

# 贪吃蛇游戏—基于面向对象编程与 Modern C++ 的课程项目

学号：1030425212 姓名：张俊哲

提交日期：2026 年 6 月 1 日

视频发布网址：<https://youtu.be/PR7z0sfKU6A?si=DdwYlhkAHj6nOxVx>

## 贪吃蛇游戏

基于面向对象编程与 Modern C++ 的课程项目

学号： 1030425212

姓名： 张俊哲

提交日期： 2026 年 6 月 1 日

## 贪吃蛇游戏项目报告

---

### C++ 程序设计课程项目报告

# 1 项目概述

## 1.1 1.1 玩法规则

本游戏是经典贪吃蛇（Snake）的命令行实现。玩家通过键盘控制一条蛇在封闭的围墙内移动。游戏的核心规则如下：

- **移动：**蛇在网格中持续前进，玩家使用 W/A/S/D 键控制方向（上/左/下/右）。
- **进食：**地图上会随机出现食物（用 \* 表示）。蛇头触碰到食物即为”吃掉”，得分增加 10 分，蛇身长度增长 1 节。
- **胜利条件：**蛇身长度达到预设目标长度（如蛇身填满整个可游玩区域）。
- **失败条件：**蛇头撞到围墙（用 # 表示），或蛇头撞到自身身体的任意一节。
- **退出：**按 ESC 键可随时退出游戏。

## 1.2 1.2 运行界面

游戏在 Windows 控制台（cmd.exe / PowerShell / Windows Terminal）中运行，使用 ANSI 转义序列实现无闪烁刷新。蛇头显示为 `o`，蛇身显示为 `o`，食物显示为 `*`，围墙显示为 `#`。每一帧在屏幕底部显示当前得分、蛇身长度和目标长度信息。

## 2 面向对象设计详解

### 2.1 2.1 类继承图

以下是本项目的类继承关系图（文字 + 箭头表示）：



## 贪吃蛇游戏项目报告

---

|                   |                     |          |
|-------------------|---------------------|----------|
| +-----+-----+     | +-----+-----+       |          |
| Snake             | Food                |          |
| -----             | -----               |          |
| - vector<Point>   | - mt19937 m_rng     |          |
| - Direction dir   | -----               |          |
| - bool growing    | + update() override |          |
| -----             | + draw() override   |          |
| + update() overr. | + respawn()         |          |
| + draw() overr.   | + isEatenBy()       |          |
| + setDirection()  | +-----+             |          |
| + grow()          |                     |          |
| + checkSelfCol()  |                     |          |
| +-----+           |                     |          |
|                   |                     |          |
| +-----+-----+     | +-----+-----+       |          |
| Board             | Game                | (不参与继承)  |
| -----             | -----               |          |
| - 2D grid         | - Board             | <-- 组合关系 |
| -----             | - Snake             |          |
| + clear()         | - Food              |          |
| + drawBorder()    | - score             |          |
| + isInside()      | -----               |          |
| + render()        | + run()             |          |
| +-----+           | +-----+             |          |

## 2.2 2.2 各类职责说明

### 2.2.1 Point（结构体）

Point 是一个简单的值类型结构体，封装二维坐标 (x, y)。它提供了 `operator==` 和 `operator!=` 用于坐标比较。Point 不参与继承体系——它作为 Snake 身体分段、Food 位置和 Board 边界判断的基础数据类型。

### 2.2.2 GameEntity（抽象基类）

GameEntity 是所有游戏实体的抽象基类。其核心职责是定义统一的接口：

- **protected 成员 m\_position:** 存储实体在当前棋盘上的位置。子类可直接访问，外部代码不可见，体现了封装原则。
- **纯虚函数 update():** 每帧更新实体状态。这是一个强制接口——任何派生类都必须实现自己的更新逻辑。
- **纯虚函数 draw():** 将实体绘制到棋盘网格上。同样为强制接口。
- **虚析构函数 virtual ~GameEntity() = default:** 确保通过基类指针删除派生类对象时，能正确调用派生类的析构函数。使用 C++11 的 `= default` 语法。

### 2.2.3 Snake（派生自 GameEntity）

Snake 类是游戏中最复杂的实体，负责管理蛇的移动、生长和碰撞检测。

- **std::vector<Point> m\_body:** 用动态数组存储蛇身各节坐标，`m_body[0]` 为蛇头。使用现代 C++ 的 `std::vector` 而非 C 风格动态数组，自动管理内存，无需手动 `new[]/delete[]`。

- **enum class Direction:** 强类型枚举表示移动方向 (Up/Down/Left/Right), 避免裸 `int` 或 `#define` 常量带来的类型安全问题。
- **m\_nextDirection 缓冲机制:** 键盘输入先存入 `m_nextDirection`, 在 `update()` 中才正式生效。这可以防止同一帧内  $180^\circ$  掉头导致蛇撞到自己。
- **生长机制:** 通过 `m_growing` 标志位实现——设为 `true` 后, 下一次 `update()` 不删除尾部, 自然实现长度 +1。
- **checkSelfCollision():** 遍历蛇身 (跳过蛇头), 检测蛇头是否与任意身体节段重合。

### 2.2.4 Food (派生自 GameEntity)

Food 类表示游戏中的食物实体。

- **std::mt19937 m\_rng:** 使用现代 C++ `<random>` 库的梅森旋转算法生成高质量随机数, 替代传统的 C 函数 `rand()/srand()`。
- **update() 为空操作:** 食物不需要每帧更新, 但必须实现此纯虚函数——这正体现了多态: Game 循环对所有实体统一调用 `update()`, 蛇会移动, 食物静止不动, 调用者无需知道具体类型。
- **respawn():** 在棋盘空白区域 (排除蛇身占据的格子) 随机选择新位置放置食物。
- **isEatenBy():** 判断蛇头坐标是否与食物位置重合。

### 2.2.5 Board (独立类, 不参与继承)

Board 类管理游戏棋盘的全部状态:

- 使用 `std::vector<std::vector<char>>` 二维向量表示网格。相比 C

的 `char[HEIGHT][WIDTH]` 固定数组或 `char**` 手动分配, `vector` 自动管理内存、边界安全, 且尺寸可在运行时动态确定。

- 提供 `clear()`、`drawBorder()`、`isInside()`、`render()` 等方法。  
`isInside()` 用于判断某个坐标是否在围墙内部。

### 2.2.6 Game (管理器类, 不参与继承)

`Game` 类是游戏的总控类, 通过组合 (composition) 持有 `Board`、`Snake` 和 `Food` 实例, 协调游戏主循环:

1. `processInput()` — 非阻塞键盘读取, 更新蛇的方向
2. `update()` — 对所有实体统一调用多态 `update()`
3. `checkCollisions()` — 检测墙壁碰撞、自碰撞、食物碰撞
4. `render()` — 清空网格 → 绘制围墙 → 多态绘制各实体 → 输出到屏幕

## 2.3 为什么选择这种继承结构

**设计思路:** 将 `Snake` 和 `Food` 的共性 (都有位置、都需要更新和绘制) 抽取到抽象基类 `GameEntity`。这种设计体现了面向对象的**开闭原则 (OCP)**——对扩展开放, 对修改封闭。如果需要添加新实体 (如障碍物 `Obstacle` 或加速道具 `PowerUp`), 只需从 `GameEntity` 派生新类, 实现 `update()` 和 `draw()`, 无需修改 `Game` 类的核心循环逻辑。

**多态的体现:** 虚函数 `update()` 和 `draw()` 在基类中声明为纯虚函数 = 0, 派生类各自重写。`Game` 类通过 `m_snake.update()` / `m_food.update()` 调用时, 编译器根据实际对象类型进行**动态绑定** (通过虚函数表 `vtable`), 自动调用正确的派生类版本。`Snake::update()` 执行移动逻辑, 而 `Food::update()`



是空操作——同样的接口，完全不同的行为。

## 2.4 对比：如果不用 OOP

假设不使用面向对象设计，将全部逻辑写在一个 main 函数中：

// 不用 OOP 的“一锅粥”伪代码：

```
int main() {  
    int snakeX[1000], snakeY[1000]; // 固定数组，浪费或溢出  
    int foodX, foodY;  
    int length = 3, direction = 0; // 裸 int 代替枚举  
    char grid[20][30]; // 固定大小  
  
    while (true) {  
        if (_kbhit()) {  
            char ch = _getch();  
            if (ch == 'w') direction = 0; // 魔法数字  
            else if (ch == 's') direction = 1;  
            // ... 更多魔法数字  
        }  
        // 移动蛇、检测碰撞、画网格……全部混在一起  
        // 200+ 行面条代码，难以理解和修改  
    }  
}
```

OOP 带来的好处：

## 贪吃蛇游戏项目报告

| 维度   | 面向过程（C 风格）          | 面向对象（本设计）                                              |
|------|---------------------|--------------------------------------------------------|
| 可维护性 | 修改蛇的行为需要改动主循环中的散落代码 | 只需修改 Snake 类，不影响其他模块                                   |
| 可扩展性 | 添加新实体需重写整个碰撞和渲染逻辑   | 派生新类，实现 <code>update()</code> / <code>draw()</code> 即可 |
| 封装性  | 全局变量泛滥，数据可被任意修改     | 成员变量 <code>private</code> ，通过公开接口安全访问                  |
| 可读性  | 数百行面条代码，逻辑交错        | 每个类职责清晰，类名即文档                                          |

### 3 现代 C++ 特征运用分析

以下选取三个最能体现”现代 C++ 风格”的代码片段进行分析。

#### 3.1 3.1 片段一：使用 `std::vector` 管理蛇身

实际代码（现代 C++ 写法）：

```
// Snake.h & Snake.cpp
class Snake : public GameEntity {
private:
    std::vector<Point> m_body;    // 动态管理蛇身，自动扩缩容
    // ...
};

void Snake::update() {
    // 使用 vector 成员函数，无需手动管理内存
    Point newHead = m_body.front();    // 获取头部
    // ... 计算新头部坐标 ...
    m_body.insert(m_body.begin(), newHead);    // 头部插入
    if (m_growing) {
        m_growing = false;
    } else {
        m_body.pop_back();    // 尾部删除
    }
}
```

如果不使用现代 C++，C 风格的写法：

```
// C 风格：手动管理动态数组

Point* m_body;           // 裸指针
int m_capacity;          // 需要额外维护容量
int m_length;

void update() {
    // 每次移动都要手动搬移所有元素
    for (int i = m_length; i > 0; i--) {
        m_body[i] = m_body[i - 1]; // 手动移位,  $O(n)$ 
    }
    m_body[0] = newHead;
    if (!m_growing) m_length--;
    // 扩容时需要手动 realloc / new[] + copy + delete[]
}
```

对比分析：

- **安全性：** `std::vector` 自动管理内存分配和释放。C 风格裸指针需要手动 `new[]/delete[]`，稍有不慎就会内存泄漏或越界访问。`vector` 的 `at()` 方法还提供边界检查，而 `[]` 运算符对裸指针无任何保护。
- **简洁性：** `insert()` / `pop_back()` 一行表达复杂的内存操作，而 C 风格需要手动移位循环。代码行数减少约 60%。
- **可读性：** `m_body.insert(...)` 的意图一目了然——“在头部插入”，而手动循环需要读者理解其在做元素搬移。

### 3.2 3.2 片段二：使用 `enum class` 和 `override` 关键字

实际代码（现代 C++ 写法）：

```
// Snake.h

class Snake : public GameEntity {
public:
    enum class Direction { Up, Down, Left, Right }; // 强类型枚举
    // ...

    void update() override; // override 关键字
    void draw(std::vector<std::vector<char>>& grid) const override;
};
```

```
// 使用 Direction 时

void Snake::setDirection(Direction dir) {
    // 编译时检查: dir 只能是 Direction 的值
    if ((m_direction == Direction::Up && dir == Direction::Down) ||
        (m_direction == Direction::Down && dir == Direction::Up) ||
        // ...
    ) return;
}
```

如果不使用现代 C++, C 风格的写法:

```
// C 风格枚举和函数覆写

enum Direction { UP, DOWN, LEFT, RIGHT }; // 全局命名空间污染

// 也可以直接: dir = 0 表示上, dir = 1 表示下
void setDirection(int dir) { // 裸 int, 任何值都能传入
    // 无法阻止 setDirection(999) 这种错误调用
    if ((m_direction == UP && dir == DOWN) || ...)
}
```

对比分析:

- **类型安全:** `enum class Direction` 是强类型的, 不能隐式转换为 `int`, 也不能与其他枚举混用。C 风格裸枚举 `UP` 是全局符号, 可能与 `windows.h` 等头文件中的宏冲突; 且枚举值可直接当 `int` 使用, 失去了类型约束。`enum class` 需要 `Direction::Up` 这种限定访问, 彻底避免命名冲突。
- **编译期保障:** `override` 关键字让编译器验证函数签名确实与基类的虚函数匹配。如果基类接口变更而派生类忘记更新, 使用 `override` 会在编译时立即报错; 不用 `override` 则会静默地创建一个新的普通函数 (而非覆写), 导致运行时调用错误。这是一个典型的”隐形 bug”来源。

### 3.3 3.3 片段三: 使用 `<random>` 和 `<chrono>` 替代 C 传统函数

实际代码 (现代 C++ 写法):

```
// Food.cpp — 随机数生成
std::mt19937 m_rng; // 梅森旋转算法
std::random_device rd;
m_rng.seed(rd());

std::uniform_int_distribution<size_t> dist(0, candidates.size() - 1);
m_position = candidates[dist(m_rng)];

// Game.cpp — 帧率控制
auto frameStart = std::chrono::steady_clock::now();
```

```
// ... 游戏逻辑 ...
```

```
auto frameEnd = std::chrono::steady_clock::now();
```

```
auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>(
    frameEnd - frameStart);
```

```
std::this_thread::sleep_for(std::chrono::milliseconds(m_frameDelayMs) - elapsed)
```

如果不使用现代 C++，C 风格的写法：

```
// C 风格随机数
```

```
srand(time(NULL)); // time_t 转 unsigned int, 精度差
```

```
int idx = rand() % candidates.size(); // 取模有偏差，低位随机性差
```

```
m_position = candidates[idx];
```

```
// C 风格帧率控制
```

```
#include <time.h>
```

```
clock_t start = clock();
```

```
// ...
```

```
clock_t end = clock();
```

```
int sleep_ms = FRAME_DELAY - (end - start) * 1000 / CLOCKS_PER_SEC;
```

```
Sleep(sleep_ms); // Windows 特定 API，不可移植
```

对比分析：

- `<random>` vs `rand()`：std::mt19937 生成的随机数质量远优于 `rand()`。`rand()` 的低位随机性差且存在取模偏差——当 `candidates.size()` 不能整除 `RAND_MAX` 时，某些位置被选中的概率更高。std::uniform\_int\_distribution 保证真均匀分布。此外 `<random>` 的种子来源 `std::random_device` 真正利用硬件熵源，而 `time(NULL)` 在同一秒内多次调用产生相同的种子。

- **<chrono> vs clock():** `clock()` 返回的是 CPU 时间而非墙上时钟时间，且精度为毫秒级，类型为 `clock_t`（通常是 `long`），语义不清。`std::chrono::steady_clock` 使用类型安全的 `time_point` 和 `duration`，精度可达纳秒，且保证单调递增（不受系统时间调整影响）。`std::this_thread::sleep_for` 是标准 C++ 的跨平台休眠函数，一次编写，在 Windows/Linux/macOS 上均可编译运行。



## 4 核心难点与解决方案

### 4.1 难点：180° 掉头导致蛇撞死自己

**问题描述：**在开发初期，我遇到了一个令游戏体验极差的 bug。当蛇向右移动时，如果玩家在一帧内快速按下 A 键（左），蛇头会立即转向，进入自己身体的位置，触发自碰撞判定而“自杀”。这是因为输入处理和移动更新在同一帧内顺序执行：`processInput()` 直接修改方向，紧接着 `update()` 就使用新方向移动蛇。

**问题定位：**通过在 `update()` 和 `setDirection()` 中添加调试输出，打印每帧的方向变化，发现方向在一个帧周期内完成了 180° 翻转：

```
Frame N:    direction = Right,  input = Left  → direction = Left    ← 问题！
Frame N+1: direction = Left,    moving Left   → head hits body[1]  ← 死亡
```

**解决方案：**引入方向缓冲（Input Buffer）机制：

1. 在 Snake 类中增加 `m_nextDirection` 成员变量，与 `m_direction` 分离。
2. `setDirection()` 不再直接修改 `m_direction`，而是将新方向写入 `m_nextDirection`。同时在写入前做合法性检查——拒绝与当前 `m_direction` 相反的输入。
3. 在 `update()` 的开始处，才将 `m_nextDirection` 正式提交到 `m_direction`。

```
void Snake::setDirection(Direction dir) {
    // 拒绝 180° 掉头
    if ((m_direction == Direction::Up    && dir == Direction::Down) ||
        (m_direction == Direction::Down && dir == Direction::Up)    ||
```

```
(m_direction == Direction::Left && dir == Direction::Right) ||
(m_direction == Direction::Right && dir == Direction::Left)) {
    return;    // 非法输入，直接忽略
}

m_nextDirection = dir;    // 缓冲新方向
}

void Snake::update() {
    m_direction = m_nextDirection;    // 正式提交
    // ... 移动逻辑 ...
}
```

**反思：**这个 bug 的根源是**状态变更时序问题**——输入和逻辑之间的”竞态”。缓冲模式是游戏开发中处理输入的经典模式：输入在任意时刻到达，但只在逻辑帧的固定节点生效。这也让我更深刻地理解了封装的价值：方向变量是 `private` 的，外部只能通过 `setDirection()` 修改，这使得添加合法性检查不影响任何外部代码。

## 5 编译与运行说明

### 5.1 运行环境

| 项目     | 说明                                                |
|--------|---------------------------------------------------|
| 操作系统   | Windows 11（或 Windows 10 及以上）                      |
| 编译器    | MinGW g++ 4.9.2 或更高版本（支持 C++14）                   |
| C++ 标准 | C++14（ <code>-std=c++14</code> ）                  |
| 依赖库    | 仅使用 C++ 标准库和 Windows <code>&lt;conio.h&gt;</code> |

提示：本项目完全使用 C++ 标准库，无任何第三方依赖。  
`<conio.h>` 仅用于非阻塞键盘输入（`_kbhit()`/`_getch()`），属于 MinGW 自带头文件。

### 5.2 编译命令

方式一：直接编译

```
g++ -std=c++14 -Wall -Wextra -O2 main.cpp Game.cpp Board.cpp Snake.cpp Food.cpp
```

方式二：使用 Makefile

```
make          # 编译
make run      # 编译并运行
make clean    # 清理编译产物
```

### 5.3 运行说明

双击 `snake_game.exe`，或在命令行中执行：

```
./snake_game.exe
```

游戏将在当前控制台窗口中启动。请确保控制台窗口尺寸足够显示完整的游戏画面（建议 80×25 字符以上）。

## 6 总结与心得

通过本次贪吃蛇课程项目，我在以下方面得到了锻炼和提升：

**面向对象设计能力：**从最初构思类的继承关系到最终实现，我体会到了“先设计接口，再实现细节”的优势。抽象基类 `GameEntity` 定义统一的 `update()`/`draw()` 接口后，`Snake` 和 `Food` 的实现可以完全独立进行，互不干扰。这种“面向接口编程”的思想，是 OOP 最有价值的核心理念之一。

**现代 C++ 的实践运用：**在项目中我有意识地用现代 C++ 特性替代传统 C 写法——`vector` 代替动态数组、`enum class` 代替裸枚举、`<random>` 代替 `rand()`、`<chrono>` 代替 `clock()`。最初觉得这些只是语法糖，但实际编码后深刻体会到它们在**安全性和可读性**上的巨大提升。尤其是 `override` 关键字，虽然只多打几个字符，却能防止一整类难以排查的运行时 bug。

**调试方法：**180° 掉头 Bug 的排查过程让我认识到，对于时序相关的 bug，简单的“加断点”往往不够——因为断点会改变时序。在代码中插入调试日志（打印关键变量状态），结合对游戏循环执行顺序的分析，才是定位此类问题的有效方法。

**项目工程化：**这次项目虽然规模不大，但遵循了头文件/源文件分离、Makefile 管理编译流程等工程实践，为今后参与更大规模的项目打下了良好基础。

## 7 附录：完整源代码

本项目全部源代码位于 `snake_game/` 目录下，文件结构如下：

```
snake_game/
|-- Point.h          -- 坐标结构体
|-- GameEntity.h    -- 抽象基类（纯虚函数接口）
|-- Snake.h         -- 蛇类头文件
|-- Snake.cpp       -- 蛇类实现（移动、碰撞、绘制）
|-- Food.h         -- 食物类头文件
|-- Food.cpp       -- 食物类实现（随机放置）
|-- Board.h        -- 棋盘类头文件
|-- Board.cpp      -- 棋盘类实现（网格管理、渲染）
|-- Game.h         -- 游戏管理类头文件
|-- Game.cpp       -- 游戏管理类实现（主循环、输入、碰撞）
|-- main.cpp       -- 程序入口
+-- Makefile       -- 编译脚本
```

### 7.0.1 Point.h

```
#ifndef POINT_H
#define POINT_H

struct Point {
    int x, y;
    Point(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const Point& o) const { return x == o.x && y == o.y; }
    bool operator!=(const Point& o) const { return !(*this == o); }
```

```
};
```

```
#endif
```

### 7.0.2 GameEntity.h

```
#ifndef GAMEENTITY_H
```

```
#define GAMEENTITY_H
```

```
#include "Point.h"
```

```
#include <vector>
```

```
class GameEntity {
```

```
protected:
```

```
    Point m_position;
```

```
public:
```

```
    explicit GameEntity(const Point& pos = Point()) : m_position(pos) {}
```

```
    virtual ~GameEntity() = default;
```

```
    virtual void update() = 0;
```

```
    virtual void draw(std::vector<std::vector<char>>& grid) const = 0;
```

```
    Point getPosition() const { return m_position; }
```

```
    void setPosition(const Point& pos) { m_position = pos; }
```

```
};
```

```
#endif
```

### 7.0.3 Snake.h

```
#ifndef SNAKE_H
#define SNAKE_H

#include "GameEntity.h"
#include <vector>

class Snake : public GameEntity {
public:
    enum class Direction { Up, Down, Left, Right };
private:
    std::vector<Point> m_body;
    Direction m_direction, m_nextDirection;
    bool m_growing;
public:
    explicit Snake(const Point& startPos, int len = 3);
    void update() override;
    void draw(std::vector<std::vector<char>>& grid) const override;
    void setDirection(Direction dir);
    void grow();
    const Point& getHeadPosition() const { return m_body.front(); }
    const std::vector<Point>& getBody() const { return m_body; }
    int getLength() const { return static_cast<int>(m_body.size()); }
    Direction getDirection() const { return m_direction; }
    bool checkSelfCollision() const;
};
```



```
#endif
```

#### 7.0.4 Snake.cpp

```
#include "Snake.h"
```

```
Snake::Snake(const Point& startPos, int len)
    : GameEntity(startPos)
    , m_direction(Direction::Right)
    , m_nextDirection(Direction::Right)
    , m_growing(false)
{
    for (int i = 0; i < len; ++i)
        m_body.emplace_back(startPos.x - i, startPos.y);
}

void Snake::update() {
    m_direction = m_nextDirection;
    Point newHead = m_body.front();
    switch (m_direction) {
        case Direction::Up:    newHead.y--; break;
        case Direction::Down:  newHead.y++; break;
        case Direction::Left:  newHead.x--; break;
        case Direction::Right: newHead.x++; break;
    }
    m_body.insert(m_body.begin(), newHead);
}
```

```
    if (m_growing) m_growing = false;
    else           m_body.pop_back();
    m_position = m_body.front();
}

void Snake::draw(std::vector<std::vector<char>>& grid) const {
    for (std::size_t i = 0; i < m_body.size(); ++i) {
        const auto& seg = m_body[i];
        if (seg.y >= 0 && seg.y < static_cast<int>(grid.size()) &&
            seg.x >= 0 && seg.x < static_cast<int>(grid[0].size()))
            grid[seg.y][seg.x] = (i == 0) ? 'O' : 'o';
    }
}

void Snake::setDirection(Direction dir) {
    if ((m_direction == Direction::Up    && dir == Direction::Down) ||
        (m_direction == Direction::Down  && dir == Direction::Up)    ||
        (m_direction == Direction::Left  && dir == Direction::Right) ||
        (m_direction == Direction::Right && dir == Direction::Left))
        return;
    m_nextDirection = dir;
}

void Snake::grow() { m_growing = true; }

bool Snake::checkSelfCollision() const {
    const auto& head = m_body.front();
```

```
    for (auto it = m_body.begin() + 1; it != m_body.end(); ++it)
        if (*it == head) return true;
    return false;
}
```

### 7.0.5 Food.h

```
#ifndef FOOD_H
#define FOOD_H

#include "GameEntity.h"
#include <random>

class Snake;

class Food : public GameEntity {
private:
    std::mt19937 m_rng;
public:
    explicit Food(int seed = 0);
    void update() override;
    void draw(std::vector<std::vector<char>>& grid) const override;
    void respawn(const std::vector<Point>& occ, int bw, int bh);
    bool isEatenBy(const Snake& snake) const;
};

#endif
```

### 7.0.6 Food.cpp

```
#include "Food.h"
#include "Snake.h"

Food::Food(int seed) : GameEntity() {
    if (seed == 0) { std::random_device rd; m_rng.seed(rd()); }
    else          m_rng.seed(static_cast<unsigned>(seed));
}

void Food::update() { /* static entity */ }

void Food::draw(std::vector<std::vector<char>>& grid) const {
    if (m_position.y >= 0 && m_position.y < static_cast<int>(grid.size()) &&
        m_position.x >= 0 && m_position.x < static_cast<int>(grid[0].size()))
        grid[m_position.y][m_position.x] = '*';
}

void Food::respawn(const std::vector<Point>& occ, int bw, int bh) {
    std::vector<Point> cand;
    for (int y = 1; y < bh - 1; ++y)
        for (int x = 1; x < bw - 1; ++x) {
            Point p(x, y);
            bool busy = false;
            for (const auto& o : occ)
                if (o == p) { busy = true; break; }
            if (!busy) cand.push_back(p);
        }
}
```

```
    }

    if (!cand.empty()) {
        std::uniform_int_distribution<size_t> dist(0, cand.size() - 1);
        m_position = cand[dist(m_rng)];
    }
}

bool Food::isEatenBy(const Snake& snake) const {
    return m_position == snake.getHeadPosition();
}
```

#### 7.0.7 Board.h

```
#ifndef BOARD_H
#define BOARD_H

#include "Point.h"
#include <vector>

class Board {
public:
    static constexpr char BORDER_CHAR = '#';
    static constexpr char EMPTY_CHAR = ' ';
    static constexpr int DEFAULT_WIDTH = 25;
    static constexpr int DEFAULT_HEIGHT = 20;
private:
    int m_width, m_height;
```

```
    std::vector<std::vector<char>> m_grid;

public:
    explicit Board(int w = DEFAULT_WIDTH, int h = DEFAULT_HEIGHT);
    void clear();
    void drawBorder();
    bool isInside(const Point& p) const;
    int getWidth() const { return m_width; }
    int getHeight() const { return m_height; }
    std::vector<std::vector<char>>& getGrid() { return m_grid; }
    const std::vector<std::vector<char>>& getGrid() const { return m_grid; }
    void render() const;
};

#endif
```

### 7.0.8 Board.cpp

```
#include "Board.h"
#include <iostream>
#include <sstream>
#include <algorithm>

Board::Board(int w, int h)
    : m_width(w), m_height(h)
    , m_grid(h, std::vector<char>(w, EMPTY_CHAR)) {}

void Board::clear() {
```

```
    for (auto& row : m_grid)
        std::fill(row.begin(), row.end(), EMPTY_CHAR);
}

void Board::drawBorder() {
    for (int x = 0; x < m_width; ++x) {
        m_grid[0][x] = BORDER_CHAR;
        m_grid[m_height - 1][x] = BORDER_CHAR;
    }
    for (int y = 0; y < m_height; ++y) {
        m_grid[y][0] = BORDER_CHAR;
        m_grid[y][m_width - 1] = BORDER_CHAR;
    }
}

bool Board::isInside(const Point& p) const {
    return p.x > 0 && p.x < m_width - 1 &&
        p.y > 0 && p.y < m_height - 1;
}

void Board::render() const {
    std::ostringstream buf;
    for (const auto& row : m_grid) {
        for (const auto& cell : row) buf << cell;
        buf << '\n';
    }
    std::cout << buf.str() << std::flush;
}
```

```
}
```

### 7.0.9 Game.h

```
#ifndef GAME_H
#define GAME_H

#include "Board.h"
#include "Snake.h"
#include "Food.h"

class Game {
private:
    Board m_board;
    Snake m_snake;
    Food m_food;
    int m_score;
    bool m_gameOver, m_won;
    int m_frameDelayMs;
    int m_targetLength;
public:
    Game(int w = 25, int h = 20, int speed = 120, int target = -1);
    void run();
private:
    void processInput();
    void update();
    void render();
};
```



```
    void checkCollisions();  
    void displayStartScreen();  
    void displayEndScreen();  
};  
  
#endif
```

#### 7.0.10 Game.cpp

```
#include "Game.h"  
#include <iostream>  
#include <thread>  
#include <chrono>  
#ifdef _WIN32  
#include <conio.h>  
#endif  
  
Game::Game(int w, int h, int speed, int target)  
    : m_board(w, h)  
    , m_snake(Point(w / 2, h / 2), 3)  
    , m_food()  
    , m_score(0), m_gameOver(false), m_won(false)  
    , m_frameDelayMs(speed), m_targetLength(target) {}  
  
void Game::run() {  
    displayStartScreen();  
    std::cout << "\033[?251";
```

```
m_food.respawn(m_snake.getBody(), m_board.getWidth(), m_board.getHeight());
std::cout << "\033[2J\033[H";
while (!m_gameOver && !m_won) {
    auto t0 = std::chrono::steady_clock::now();
    processInput();
    update();
    checkCollisions();
    render();
    auto t1 = std::chrono::steady_clock::now();
    auto elap = std::chrono::duration_cast<std::chrono::milliseconds>(t1 - t0);
    auto rem = std::chrono::milliseconds(m_frameDelayMs) - elap;
    if (rem.count() > 0) std::this_thread::sleep_for(rem);
}
std::cout << "\033[?25h";
displayEndScreen();
}

void Game::processInput() {
#ifdef _WIN32
    while (_kbhit()) {
        char ch = _getch();
        switch (ch) {
            case 'w': case 'W': m_snake.setDirection(Snake::Direction::Up); break;
            case 's': case 'S': m_snake.setDirection(Snake::Direction::Down); break;
            case 'a': case 'A': m_snake.setDirection(Snake::Direction::Left); break;
            case 'd': case 'D': m_snake.setDirection(Snake::Direction::Right); break;
            case 27: m_gameOver = true; break;
        }
    }
#endif
}
```

```
        }

    }

#endif

}

void Game::update() {
    m_snake.update();
    m_food.update();
}

void Game::render() {
    m_board.clear();
    m_board.drawBorder();
    m_snake.draw(m_board.getGrid());
    m_food.draw(m_board.getGrid());
    std::cout << "\033[H";
    m_board.render();
    std::cout << "Score: " << m_score
        << " | Length: " << m_snake.getLength()
        << " | Target: "
        << (m_targetLength > 0 ? std::to_string(m_targetLength)
            : std::string("Fill board"))
        << std::endl;
    std::cout << "W/A/S/D to move | ESC to quit" << std::endl;
}

void Game::checkCollisions() {
```

```
const Point& head = m_snake.getHeadPosition();
if (!m_board.isInside(head)) { m_gameOver = true; return; }
if (m_snake.checkSelfCollision()) { m_gameOver = true; return; }
if (m_food.isEatenBy(m_snake)) {
    m_snake.grow();
    m_score += 10;
    m_food.respawn(m_snake.getBody(), m_board.getWidth(), m_board.getHeight());
    if (m_targetLength > 0) {
        if (m_snake.getLength() >= m_targetLength) m_won = true;
    } else {
        int area = (m_board.getWidth() - 2) * (m_board.getHeight() - 2);
        if (m_snake.getLength() >= area) m_won = true;
    }
}
}

void Game::displayStartScreen() {
    std::cout << "=====\n"
        << "          Welcome to SNAKE!\n"
        << "=====\n\n"
        << "Controls:  W=Up   S=Down  A=Left  D=Right\n"
        << "          ESC = Quit\n\n"
        << "Eat food (*) to grow. Avoid walls (#) and yourself!\n\n";
    if (m_targetLength > 0)
        std::cout << ">>> Reach length " << m_targetLength << " to win! <<<\n\n"
    else
        std::cout << ">>> Fill the entire board to win! <<<\n\n";
}
```

```
    std::cout << "Press any key to start...";
#ifdef _WIN32
    _getch();
#endif
}

void Game::displayEndScreen() {
    std::cout << "\033[2J\033[H";
    std::cout << "=====\n";
    if (m_won)
        std::cout << "                YOU WIN!\n";
    else
        std::cout << "                GAME OVER\n";
    std::cout << "=====\n\n"
        << "Final Score:  " << m_score << "\n"
        << "Final Length: " << m_snake.getLength() << "\n\n"
        << "Press any key to exit...";
#ifdef _WIN32
    _getch();
#endif
}
```

#### 7.0.11 main.cpp

```
#include "Game.h"
#include <iostream>
```

```
int main() {
    try {
        Game game(30, 20, 100, -1);
        game.run();
    } catch (const std::exception& e) {
        std::cerr << "Fatal error: " << e.what() << std::endl;
        return 1;
    }
    return 0;
}
```

#### 7.0.12 Makefile

```
CXX      = g++
CXXFLAGS = -std=c++14 -Wall -Wextra -O2
TARGET   = snake_game.exe
SRCS     = main.cpp Game.cpp Board.cpp Snake.cpp Food.cpp
OBJS     = $(SRCS:.cpp=.o)

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CXX) $(CXXFLAGS) -o $@ $^

%.o: %.cpp
    $(CXX) $(CXXFLAGS) -c $< -o $@
```

## 贪吃蛇游戏项目报告

---

clean:

```
rm -f $(OBJJS) $(TARGET)
```

run: \$(TARGET)

```
./$(TARGET)
```

---

本报告及其附带的全部源代码由作者独立完成。